

Introduction to MACHINE LEARNING: module 1.

Fosca Giannotti
Roberto Pellungrini



SCUOLA
NORMALE
SUPERIORE

Supervised Learning

Deep Learning with Recurrent Neural Networks: architecture and exemplar usecase

Table of contents

1. Introduction
2. RNN
3. LSTM
4. GRU
5. Time Series
6. Notebook
7. Conclusion

Intro

Dealing with sequential data

SEQUENTIAL DATA: any kind of data where the order of the observations matters.

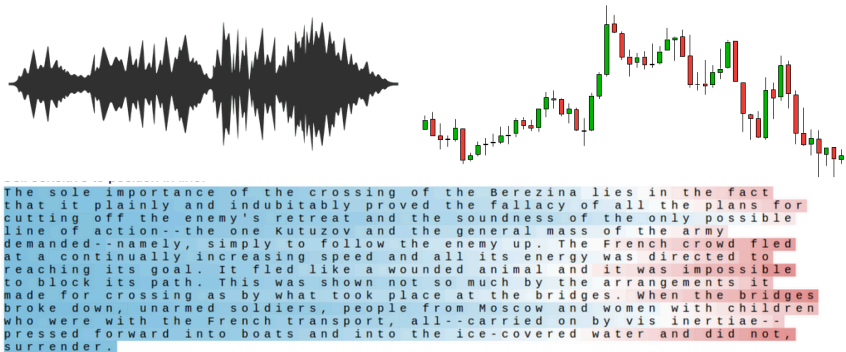


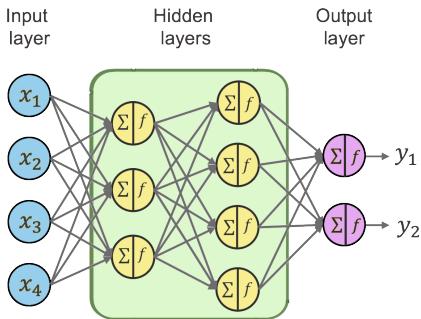
Figure 1: Examples of audio data, financial time series, text.

Dealing with sequential data

	x - input		y - output
Speech recognition		→	"Fuzzy Wuzzy was a bear. Fuzzy Wuzzy had no hair."
Music generation	∅	→	
Sentiment classification	"Decent effort. The plot could have been better."	→	
DNA sequence analysis	ACTGTACCCATGTGACTGCCC	→	ACTGTACCCATGTGACTGCCC
Machine translation	"El que no arriesga, no gana."	→	"If you don't take risks, you cannot win."
Video activity recognition		→	Running
Name entity recognition	"Ygritte says Jon Snow knows nothing."	→	"Ygritte says Jon Snow knows nothing."

Why not vanilla neural networks?

The key parts of a vanilla neural network are: the **input**, the **output**, the **hidden layers**, and the **weights** between the nodes. Vanilla Neural Networks take a fixed-sized input and produce a fixed-sized output.



$$h = f_{W_{xh}}(x)$$

$$y = f_{W_{hy}}(h)$$

- x : input
- h : hidden state
- f_W : some activation function with weights W
- y : output

Why not vanilla neural networks?

Vanilla neural networks have some problems in dealing with sequential data:

- they **don't share parameters**, i.e. can't learn similar features in different parts of the sequential data;
- they can't handle **different sized data**;
- they **don't consider the sequential structure** of the data: data points depend upon previous data points.

Yesterday I ate pizza

I ate pizza yesterday

What did you eat yesterday?

RNN

Recurrent neural networks (RNNs) [1] are a class of neural networks that allow **previous outputs** to be used as **inputs** of the network. They have a sort of “*memory*” that stores information about the past to inform predictions about the future.

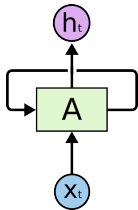


Figure 2: “rolled” RNN

$$h_t = f_W(h_{t-1}, x_t)$$

- x_t : input at time-step t
- A : memory cell
- h_t : new state
- h_{t-1} : old state
- f_W : some activation function with parameters W

The RNN can be thought of as **multiple copies of the same network**, each passing a message to a successor.

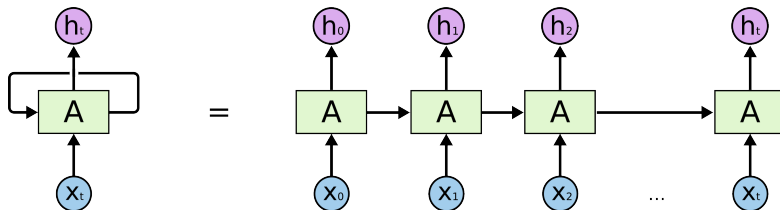
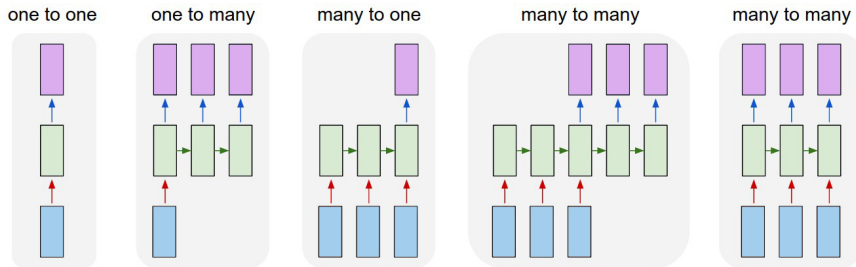


Figure 3: “unrolled” RNN

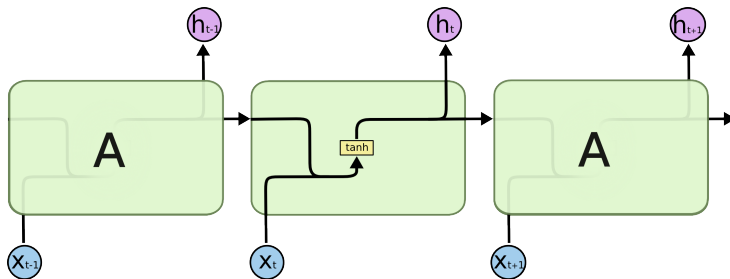
Different applications

There are different kinds of possible network structures depending on the task at hand.



- **one to one:** vanilla NN;
- **one to many:** e.g. image captioning (image $\rightarrow$ sequence of words);
- **many to one:** e.g. sentiment classification (sequence of words $\rightarrow$ sentiment);
- **many to many:** e.g. machine translation (sequence of words $\rightarrow$ sequence of words).

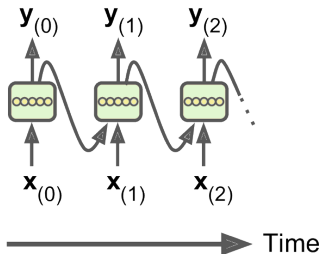
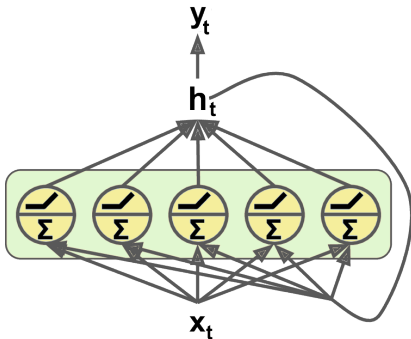
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t) = \tanh(W_h[h_{t-1}, x_t])$$



Output

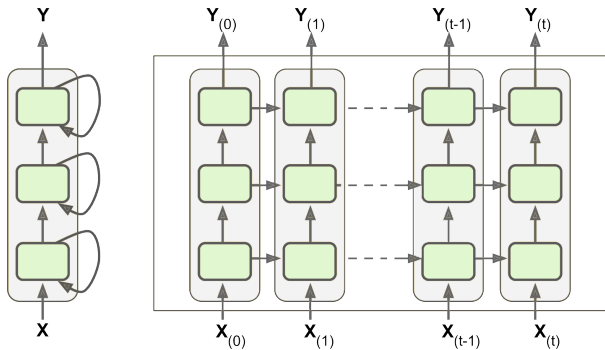
You can have **multiple neurons** inside the memory cell, like in a standard neural network layer. The output of a RNN is:

$$y_t = W_{hy}h_t$$



Deep RNNs

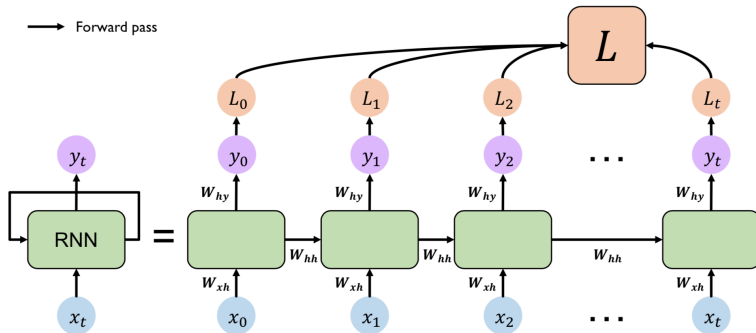
A **deep RNN** can be built by stacking multiple memory cells.



Training: forward pass

In the **forward pass**, the output, y , of the network is compared to the ground truth, o .

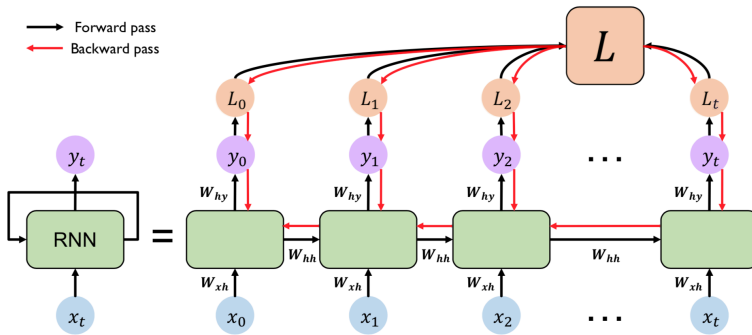
$$L(y, o) = \sum_{t=0}^T L_t(y_t, o_t)$$



Training: backward pass

In order to learn, RNNs use the so-called **backpropagation through time**.

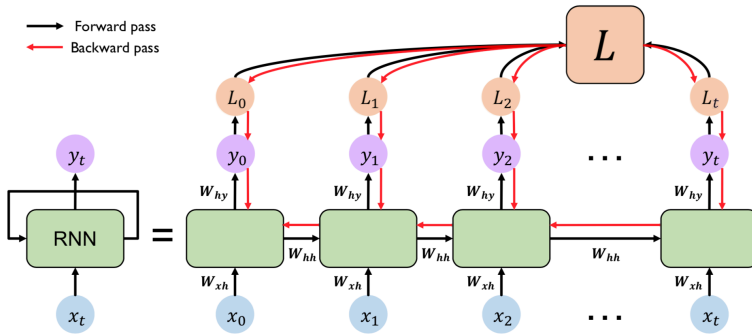
$$\frac{\partial L}{\partial W} = \sum_{t=0}^T \frac{\partial L_t}{\partial W} = \sum_{t=0}^T \sum_{k=0}^t \frac{\partial L_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W}$$



Training: backward pass

In order to learn, RNNs use the so-called **backpropagation through time**.

$$\frac{\partial L}{\partial W} = \sum_{t=0}^T \frac{\partial L_t}{\partial W} = \sum_{t=0}^T \sum_{k=0}^t \frac{\partial L_t}{\partial h_t} \frac{\partial h_t}{\partial h_k} \frac{\partial h_k}{\partial W}$$



Gradients flow

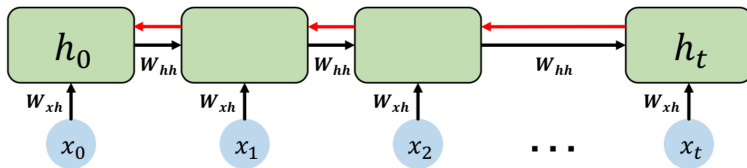
Given that the weights in the RNN are shared, backpropagating the gradients involves **many matrix multiplications** with W .

$$\frac{\partial h_t}{\partial h_k} = \prod_{j=k+1}^t \frac{\partial h_{j+1}}{\partial h_j} = \prod_{j=k+1}^t W^T \times \text{diag}[f'(h_{j-1})]$$

The gradient is a recursive product of hidden activation gradients. So if:

$$\left\| \frac{\partial h_{j+1}}{\partial h_j} \right\| < 1 \quad \text{vanishing gradients}$$

$$\left\| \frac{\partial h_{j+1}}{\partial h_j} \right\| > 1 \quad \text{exploding gradients}$$

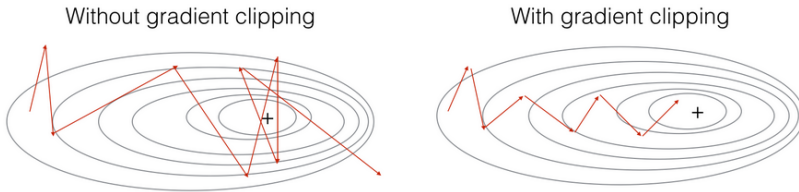


Exploding Gradients

Exploding Gradients are a problem because if the gradient value grows extremely large, it causes an **overflow** (i.e. NaN values). Exploding gradients are easily detectable at runtime and can be solved by using **gradient clipping**:

$$\text{Given } g = \frac{\partial L}{\partial W}, \text{ if } \|g\| \geq \text{threshold}, \text{ then } g = g \frac{\text{threshold}}{\|g\|}$$

In practice, gradients are scaled back to a small number whenever they reach a certain threshold, without changing their direction.



Vanishing Gradients

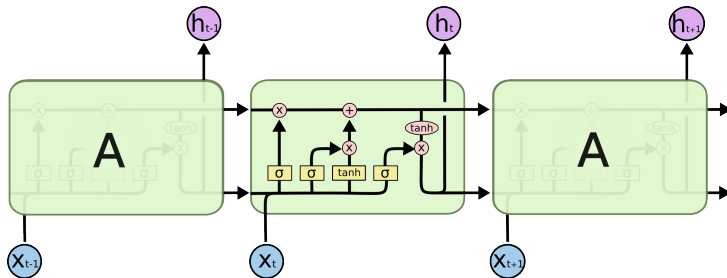
When the gradient value goes to zero, it will be increasingly hard to update the network weights, $W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial L}{\partial W}$, **reducing the learning quality of the model for far-away observations** in the input data.

Possible solutions are:

- smart **weights** initialization (identity matrix);
- use the Rectified Linear Units (**ReLU**) instead of the sigmoid or tanh functions (the derivative for the ReLU is either 0 or 1);
- **truncated** backpropagation: truncating the time steps at a computationally convenient window size;
- **gating**.

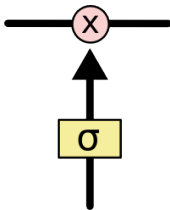
LSTM

The long short-term memory (LSTM) network [2] is a recurrent architecture composed of a **cell state**, an **input gate**, an **output gate** and a **forget gate**. The cell state can remember values over long time intervals, and the three gates regulate the flow of information into and out of the cell. LSTMs can model **long-term dependencies**!



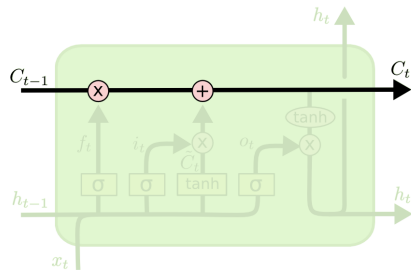
Gates

Gates are composed by a **sigmoid layer** and a **pointwise multiplication** operation*.



$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \odot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} \\ a_{21}b_{21} & a_{22}b_{22} \end{bmatrix}$$

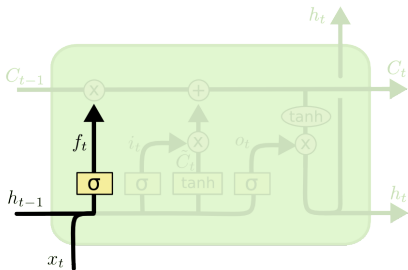
*Hadamard product



Gates have the ability to remove or add information to the cell state, in which the **information can flow with only some minor linear interactions.**

Forget Gate

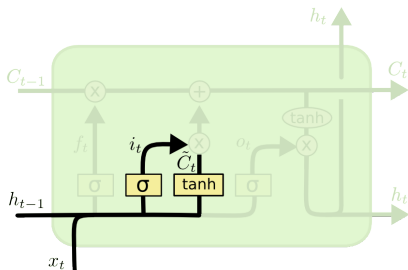
The **forget gate** decides which parts of the long-term state C_{t-1} should be erased, by looking at h_{t-1} and x_t .



$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

New Memory Generation - Input Gate

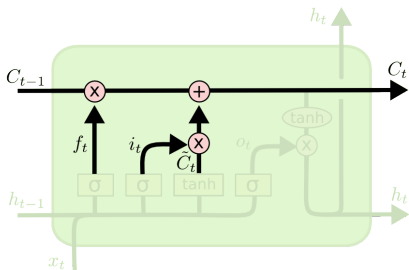
To decide what new information to store in the cell state, there are two parts: an **input gate** that decides which values to update, and a **tanh layer** that creates a vector of new candidate values $\tilde{C}_t$ that could be added to C_t .



$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

Cell state update

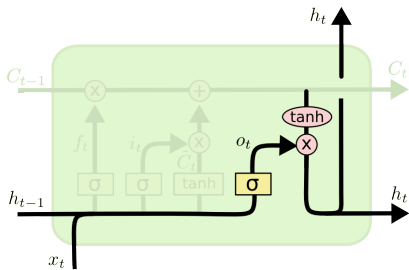
The old cell state C_{t-1} is updated into the cell state C_t by first multiplying the old state by f_t , and then adding $i_t \odot \tilde{C}_t$. The first half of the operation forgets, while the second half adds the new scaled candidate values.



$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

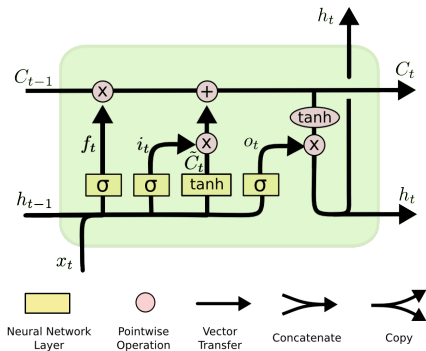
Output Gate

The **output gate** decides what parts of the cell state are going to be output. The cell state goes through a **pointwise tanh** and is multiplied by the output of the output gate.



$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$



$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C)$$

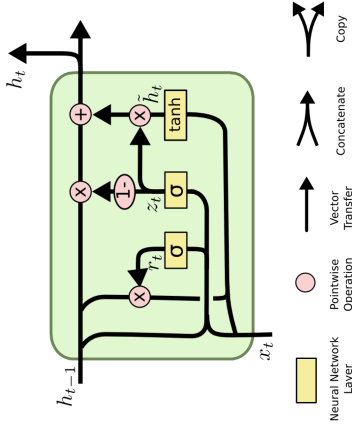
$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

$$h_t = o_t \odot \tanh(C_t)$$

GRU

The Gated Recurrent Unit (GRU) [3] has an **update gate**, z_t , which combines the forget gate and the input gate, and a **reset gate**, r_t . It also **merges the cell state and hidden states**. GRUs are becoming increasingly popular because of their simplicity w.r.t. LSTMs.



$$z_t = \sigma(W_z[h_{t-1}, x_t])$$

$$r_t = \sigma(W_r[h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W[r_t \odot h_{t-1}, x_t])$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Time Series

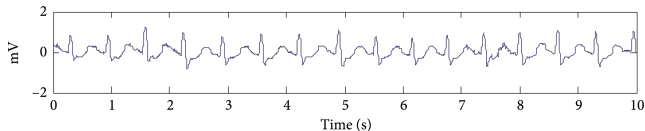
Time Series

TIME SERIES: a univariate time series $x = \{t_1, t_2, \dots, t_m\} \in \mathbb{R}^m$ is an ordered set of m real-valued observations, sampled at equal time intervals.

TIME SERIES DATASET: a univariate time series dataset X is a set of n time series, $X = \{x_1, x_2, \dots, x_n\} \in \mathbb{R}^{n \times m}$.

TIME SERIES CLASSIFICATION: given a dataset X with a vector of assigned labels (or classes), $Y = \{y_1, y_2, \dots, y_n\} \in \mathbb{N}^n$, Time Series Classification is the task of training a function or mapping f from the space of possible inputs X to a probability distribution over the class variable values Y .

TIME SERIES FORECASTING: Time series forecasting models predict future observations by looking at past observations.



Notebook

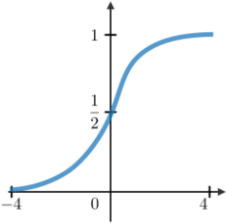
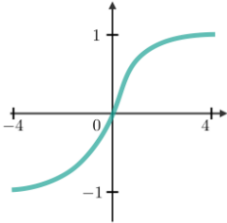
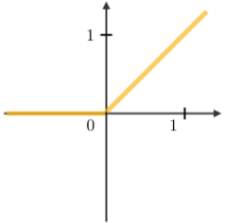
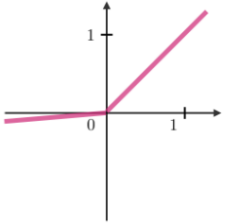
<https://colab.research.google.com/>

Conclusion

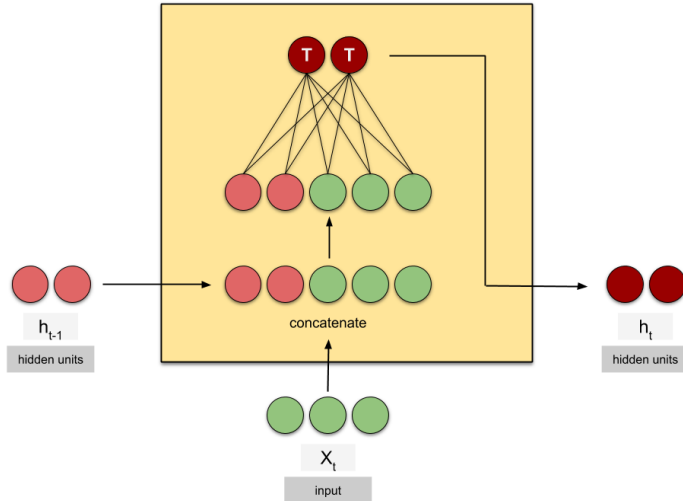
Take home messages:

- sequential data is any kind of data where the order of the observations matter;
- standard neural network are not well suited to solve problems regarding sequential data;
- RNNs can deal with sequential data because (i) they can pass information from the past to the future, (ii) can handle inputs of different lengths, (iii) share weights between time-steps;
- vanilla RNNs struggle with long sequences;
- LSTMs, GRUs implement gates, solving some of the problems of vanilla RNNs.

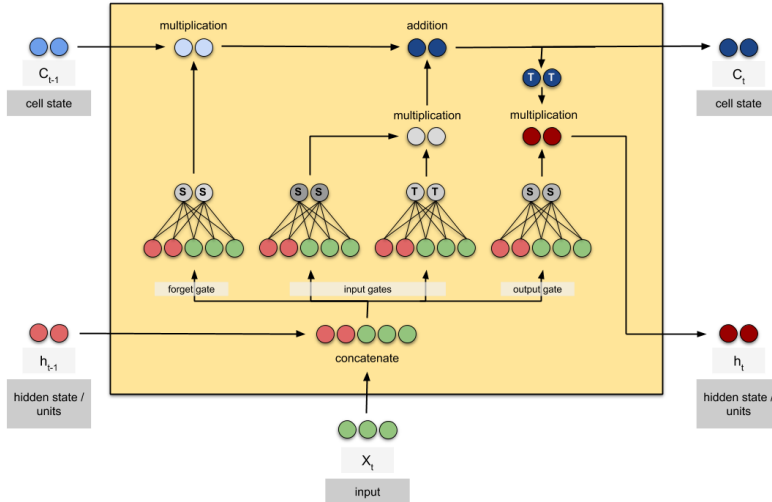
Activation Functions

Sigmoid	Tanh	ReLU	Leaky ReLU
$g(z) = \frac{1}{1 + e^{-z}}$	$g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$	$g(z) = \max(0, z)$	$g(z) = \max(\epsilon z, z)$ with $\epsilon \ll 1$
			

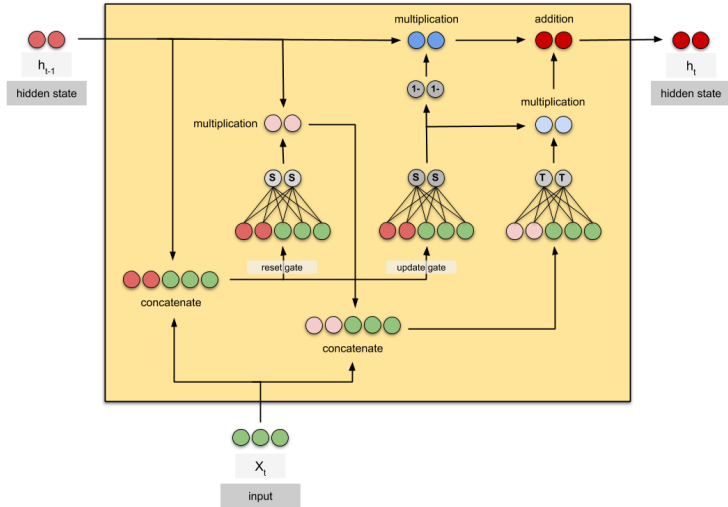
RNN detail



LSTM detail



GRU detail



References i

-  L. C. Jain and L. R. Medsker, *Recurrent Neural Networks: Design and Applications*, 1st ed. USA: CRC Press, Inc., 1999.
-  S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 11 1997. [Online]. Available: <https://doi.org/10.1162/neco.1997.9.8.1735>
-  K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder-decoder for statistical machine translation,” *arXiv preprint arXiv:1406.1078*, 2014.
-  A. Géron, *Neural networks and deep learning*. O'Reilly, 2018.
-  “Recurrent neural networks (rnn) with keras.” [Online]. Available: <https://www.tensorflow.org/guide/keras/rnn>



“Rnn - pytorch 1.10 documentation.” [Online]. Available:
<https://pytorch.org/docs/stable/generated/torch.nn.RNN.html>



I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016,
<http://www.deeplearningbook.org>.